Universidad Católica "Nuestra Señora de la Asunción"

Campus Alto Paraná – Facultad de Ciencias y Tecnología

AL2026-1S – Álgebra Lineal Computacional

Proyecto 1: Construcción de Mapas Robóticos con Datos LiDAR

> **Adaptado del proyecto original:**
> **University of Michigan – ROB 101: Computational Linear Algebra**
> **Project 1: Robotic Mapping with LiDAR Data**
> https://grizzle.robotics.umich.edu/education/rob101

Bienvenido al mundo de las Transformaciones Lineales

En este proyecto exploraremos cómo el Álgebra Lineal permite resolver problemas reales de robótica mediante transformaciones matriciales. A través del procesamiento de datos obtenidos por un sensor LiDAR, experimentaremos el enorme poder de las matrices para manipular y transformar grandes (¡enormes!) cantidades de datos de manera eficiente.

El objetivo es comprender cómo las transformaciones afines (traslación, rotación y escalamiento) permiten construir mapas del entorno a partir de millones de mediciones realizadas por un robot móvil.

Código de Honor Académico

- Puede realizar consultas generales sobre los conceptos teóricos y metodológicos del proyecto.
- Puede solicitar ayuda cuando existan problemas técnicos relacionados con el entorno de ejecución o con los archivos de trabajo.
- Las consultas específicas sobre la resolución de ejercicios deberán realizarse a través de los mecanismos de apoyo definidos por la cátedra.
- No debe compartir código fuente, soluciones completas ni resultados finales con otros estudiantes.
- El trabajo debe realizarse de forma individual.

- La entrega de este proyecto constituye una declaración de que el trabajo presentado es producto de su propio esfuerzo académico, respetando las normas éticas y de integridad establecidas por la Universidad Católica "Nuestra Señora de la Asunción".

Objetivos de Aprendizaje

Al finalizar este proyecto el estudiante será capaz de:

- Aplicar transformaciones lineales mediante multiplicación matricial.
- Utilizar coordenadas homogéneas para representar transformaciones afines.
- Implementar traslaciones, rotaciones y escalamientos en dos dimensiones.
- Comprender la construcción de transformaciones afines compuestas.
- Procesar nubes de puntos provenientes de sensores LiDAR.
- Transformar datos desde un sistema de coordenadas local a un sistema de coordenadas global.
- Comprender el papel del Álgebra Lineal en aplicaciones modernas de robótica móvil, percepción y mapeo.

Referencia Bibliográfica

University of Michigan. (2022). *ROB 101: Computational Linear Algebra – Project 1: Robotic Mapping with LiDAR Data*. Michigan Robotics Institute. Material adaptado con fines educativos para la asignatura **AL2026-1S – Álgebra Lineal Computacional**, Facultad de Ciencias y Tecnología, Universidad Católica "Nuestra Señora de la Asunción", Campus Alto Paraná.

In [ ]:
```
using Plots
```

Introducción a las actividades

En este proyecto trabajaremos con datos reales provenientes de sensores LiDAR utilizados en robótica móvil. A lo largo de las distintas tareas aplicaremos conceptos fundamentales de Álgebra Lineal para representar información geométrica, transformar sistemas de coordenadas y construir mapas tridimensionales a partir de millones de mediciones.

El proyecto está organizado de manera progresiva. Cada tarea introduce una nueva herramienta matemática que será utilizada posteriormente en problemas de mayor complejidad. Se recomienda completar las actividades en el orden propuesto.

Tarea 0 — Repaso computacional

Antes de comenzar con el proyecto principal, realizaremos una breve etapa de familiarización con el entorno de trabajo.

En esta sección repasaremos la representación de vectores y matrices, la indexación de arreglos, las operaciones matriciales básicas y las herramientas de visualización que utilizaremos durante el resto del proyecto.

El objetivo es asegurar que todos los estudiantes dispongan de las bases necesarias para abordar las actividades posteriores con autonomía.

Tarea 1 — Transformaciones Afines en 2D

En esta tarea estudiaremos cómo las transformaciones geométricas pueden representarse mediante matrices.

Implementaremos traslaciones, rotaciones, escalamiento y transformaciones afines utilizando coordenadas homogéneas. Estas herramientas constituyen la base matemática utilizada en robótica, visión artificial, gráficos por computadora y sistemas de navegación.

El objetivo principal es comprender cómo una secuencia de operaciones geométricas puede expresarse mediante multiplicación matricial.

Tarea 2 — Corrección de una nube de puntos 2D

Una vez comprendidas las transformaciones afines, las aplicaremos sobre una nube de puntos bidimensional que representa una imagen distorsionada.

Utilizando una transformación homogénea conocida, corregiremos la geometría de los datos para recuperar la imagen original.

Esta actividad muestra cómo las herramientas desarrolladas en la tarea anterior pueden utilizarse para resolver problemas reales de percepción y procesamiento de datos.

Tarea 3 — Construcción de mapas LiDAR en 3D

En la última etapa del proyecto trabajaremos con datos reales obtenidos por un sensor LiDAR montado sobre un robot móvil.

Cada medición se encuentra inicialmente expresada en el sistema de coordenadas local del sensor. Nuestro objetivo será transformar dichas mediciones a un sistema de referencia global y fusionar múltiples escaneos para construir un mapa tridimensional coherente del entorno.

Esta tarea reproduce, de manera simplificada, uno de los problemas fundamentales de la robótica moderna: la construcción de mapas a partir de sensores embarcados en plataformas móviles.

Reflexión final

Al finalizar el proyecto habremos utilizado vectores, matrices, coordenadas homogéneas, transformaciones afines y cambios de sistema de referencia para resolver un problema auténtico de robótica.

Más allá de la programación o de la visualización de datos, el propósito central del trabajo es comprender cómo el Álgebra Lineal constituye el lenguaje matemático que permite representar, transformar e interpretar información en sistemas tecnológicos complejos.

==========

TAREA 0 — REPASO COMPUTACIONAL

==========

Esta tarea constituye un repaso de operaciones básicas con vectores y matrices. Todos estos conceptos ya fueron trabajados en clases, prácticas y ejercicios previos, por lo que puede considerarse una actividad de nivelación.

Ninguna actividad anterior a la Tarea 1 forma parte de la evaluación del proyecto.

Matrices

Sea A una matriz de dimensiones $n \times m$:

- `size(A,1)` devuelve n, el número de filas de la matriz.
- `size(A,2)` devuelve m, el número de columnas de la matriz.
- `size(A)` devuelve el par (n, m), indicando primero el número de filas y luego el número de columnas.
- `length(A)` devuelve $n \times m$, es decir, el número total de elementos contenidos en la matriz.

Vectores

Sea v un vector de dimensión n:

- `length(v)` devuelve n, el número de elementos del vector.
- Aunque es posible aplicar la función `size()` a un vector, generalmente no es la opción recomendada.
- Para vectores, se prefiere utilizar `length()` porque expresa directamente la cantidad de elementos que contiene.

Importante

En la práctica, rara vez se necesita conocer el número total de elementos de una matriz.

La mayoría de las operaciones requieren conocer:

- el número de filas y columnas;
- únicamente el número de filas; o
- únicamente el número de columnas.

Por este motivo, las funciones `size(A,1)` y `size(A,2)` suelen ser mucho más útiles que `length(A)` cuando se trabaja con matrices.

In [ ]:
```
# Ejecute esta celda. Genera algunos datos de ejemplo para practicar.

using Random

# Fijamos la semilla del generador de números aleatorios para que
# todos obtengamos exactamente los mismos resultados.
Random.seed!(101)

# Matriz aleatoria de 3 filas y 4 columnas
A = randn(3,4)

# Matriz aleatoria de 4 filas y 10 columnas
B = randn(4,10)

# Vector columna de dimensión 3
v3 = [1; 2; 3.0]

# Vector columna de dimensión 4
v4 = [1; 2; 3; 4.0]

# Observación:
#
# Las dimensiones fueron elegidas deliberadamente para que algunas
# operaciones matriciales sean válidas y otras no.
#
# A es una matriz de tamaño 3×4
# B es una matriz de tamaño 4×10
#
# Por ejemplo:
#   A*B  -> está definida (3×4)(4×10) = 3×10
#
# mientras que:
#   B*A  -> no está definida
```

```
#
# porque las dimensiones internas no coinciden.
#
# Utilice estos datos para experimentar con productos matriz-matriz,
# matriz-vector y operaciones básicas de Álgebra Lineal.
```

In [ ]:
```
# número de filas y columnas de una matriz
#
# Descomente la siguiente línea y complete el comando para obtener
# el número de filas y columnas de la matriz A

# nRows, nCols = ???


# Verificación rápida
if [nRows nCols] == [3 4]
    println("¡Correcto!")
elseif [nRows nCols] == [4 3]
    println("Eso sería [nCols nRows]. Inténtelo nuevamente.")
else
    println("No está claro qué salió mal. Inténtelo otra vez utilizando l
end
```

In [ ]:
```
# Si SOLO desea obtener el número de columnas de una matriz y no necesita
# conocer el número de filas,
#
# descomente la siguiente línea y complete el comando para obtener
# ÚNICAMENTE el número de columnas de la matriz B.
#
# nColsB = ??

# Verificación rápida
if nColsB == 10
    println("Muy bien. Va por buen camino.")
elseif nColsB == 40
    println("Parece que utilizó la función length(), lo cual es incorrect
elseif nColsB == 3
    println("Incorrecto. Ha confundido filas con columnas.")
else
    println("No podemos identificar el origen del error. Inténtelo nuevam
end

if @isdefined(nRowsB)
    println("Parece que utilizó nRowsB, nColsB = size(B). En este ejercic
end
# Recordatorio:
#
# size(B,1) devuelve el número de filas.
# size(B,2) devuelve el número de columnas.
# length(B) devuelve el número total de elementos.
#
# Si B es una matriz de tamaño 4×10:
#
# size(B,1) = 4
# size(B,2) = 10
# length(B) = 40
#
# En este ejercicio solamente nos interesa conocer
# el número de columnas de B.
```

In [ ]:
```
# Sumar un vector compatible a CADA una de las columnas de una matriz
#
# A continuación se muestran tres métodos para sumar el vector v3
# a cada columna de la matriz A.

C = copy(A)

# Método 1: fuerza bruta
C[:,1] = C[:,1] + v3
C[:,2] = C[:,2] + v3
C[:,3] = C[:,3] + v3
C[:,4] = C[:,4] + v3

display(A)
display(C)
display(C - A)

# La matriz A tiene tamaño 3×4 y el vector v3 tiene dimensión 3.
#
# Cada columna de A es un vector de dimensión 3:
#
# A = [a₁ a₂ a₃ a₄]
#
# donde:
#
# a₁, a₂, a₃ y a₄ ∈ R³
#
# Al sumar v3 a cada columna obtenemos:
#
# C = [a₁+v3  a₂+v3  a₃+v3  a₄+v3]
#
# Observe que:
#
# C - A
#
# devuelve una matriz cuyas columnas son exactamente v3.
#
# Este ejemplo ilustra una idea fundamental del Álgebra Lineal:
# una misma operación vectorial puede aplicarse simultáneamente
# a muchas columnas de una matriz.
#
# Más adelante utilizaremos esta misma idea para trasladar miles
# o millones de puntos de una nube LiDAR mediante una única
# operación matricial.
```

In [ ]:
```
C = copy(A)

# Método 2: utilizar un bucle for
for i = 1:size(C,2)
    C[:,i] = C[:,i] + v3   # C[:,i] selecciona todas las filas y la colum
end

display(A)
display(C)
display(C - A)

# En lugar de escribir una instrucción para cada columna,
# utilizamos un bucle for para recorrer automáticamente
# todas las columnas de la matriz.
```

```
#
# size(C,2) devuelve el número de columnas de C.
#
# Si C es una matriz 3×4:
#
# i = 1, 2, 3, 4
#
# En cada iteración se suma el vector v3 a la columna i:
#
# C[:,i] = C[:,i] + v3
#
# donde:
#
# C[:,i]
#
# significa:
#
# ":"  -> todas las filas
# "i"  -> columna i
#
# Este enfoque es más escalable que el Método 1.
#
# Si la matriz tuviera 100 columnas, el mismo código
# seguiría funcionando sin modificaciones.
#
# En aplicaciones reales de robótica y procesamiento
# de datos es común trabajar con miles o millones de
# vectores almacenados como columnas de una matriz.
#
# Por ello, automatizar operaciones mediante bucles
# constituye una herramienta fundamental.
#
# Observe nuevamente que:
#
# C - A
#
# contiene el efecto de la transformación aplicada,
# es decir, el vector v3 sumado a cada columna.
```

In [ ]:

```
# Este es el método más compacto y normalmente el más utilizado en Julia.
#
# La operación:
#
# C .+ v3
#
# indica que Julia debe sumar el vector v3 elemento a elemento
# sobre toda la matriz C utilizando broadcasting.
#
# Conceptualmente es equivalente a:
#
# for i = 1:size(C,2)
#     C[:,i] = C[:,i] + v3
# end
#
# pero Julia realiza la operación de manera más eficiente y con
# una sintaxis mucho más compacta.
#
# El punto (.) es fundamental.
#
# Correcto:
```

```
#
# C .+ v3
#
# Incorrecto:
#
# C + v3
#
# Sin el punto, Julia intentará realizar una suma matricial
# convencional entre objetos con dimensiones incompatibles,
# produciendo un error.
#
# Desde el punto de vista geométrico, estamos trasladando todas
# las columnas de la matriz mediante el mismo vector v3.
#
# Esta idea aparece constantemente en:
#
# - procesamiento de imágenes;
# - visión artificial;
# - robótica móvil;
# - aprendizaje automático;
# - procesamiento de nubes de puntos LiDAR.
#
# Más adelante utilizaremos exactamente este principio para
# trasladar simultáneamente miles o millones de puntos mediante
# operaciones matriciales vectorizadas.
```

In [ ]:

```
# Utilice su método favorito para sumar el vector v4 a cada columna
# de la matriz B.
#
# Guarde el resultado en la matriz C.

C = copy(B)

# Método recomendado: broadcasting
C = C .+ v4

# Las verificaciones asumen que el resultado está almacenado en C
E = C - B; test = true
for i = 1:length(v4)
    for j = 1:size(E,2)
        test = test & isapprox(E[i,j], i, atol = 1e-2)
    end
end

if test == true
    println("Buen trabajo")
else
    println("Inténtelo nuevamente. Necesitará dominar esta operación en e
end

# B es una matriz de tamaño 4×10.
#
# v4 es un vector de dimensión 4:
#
# v4 = [1; 2; 3; 4]
#
# Como cada columna de B tiene exactamente 4 elementos,
# podemos sumar v4 a todas las columnas simultáneamente.
#
# Geométricamente estamos trasladando cada columna de B
```

```
# mediante el mismo vector v4.
#
# La operación:
#
# C = C .+ v4
#
# utiliza broadcasting para aplicar la misma suma a todas
# las columnas de la matriz.
#
# Esta idea será utilizada más adelante para transformar
# grandes conjuntos de puntos mediante operaciones matriciales
# eficientes.
```

In [ ]:

```
# Multiplicar una matriz compatible por cada columna de una segunda matri
#
# A continuación se muestran distintas formas de multiplicar cada
# columna de la matriz B por la matriz A y almacenar los resultados
# en una matriz C de dimensiones adecuadas.


# Método 1: utilizando un bucle for

nRowsA = size(A,1)
nColsB = size(B,2)

C = zeros(nRowsA, nColsB)   # matriz de ceros del tamaño correcto

for k = 1:size(B,2)
    v = B[:,k]      # extraer la columna k de B
    Av = A * v      # multiplicar A por la columna k
    C[:,k] = Av     # almacenar el resultado en la columna k de C
end


# Método 1-a: una pequeña variación del método anterior

D = zeros(nRowsA, nColsB)   # matriz de ceros del tamaño correcto

for k = 1:size(B,2)
    D[:,k] = A * B[:,k]     # realizar todo en una sola instrucción
end


# Método 1-b: otra variación

E = Array{Float64}(undef, nRowsA, nColsB)  # matriz vacía del tamaño corr
                                           # creada utilizando undef

for k = 1:size(B,2)
    E[:,k] = A * B[:,k]     # realizar todo en una sola instrucción
end

# ¿Son iguales los resultados?

display(C - D)
display(C - E)

# Recordemos las dimensiones:
#
# A es una matriz 3×4
```

```
# B es una matriz 4×10
#
# Por lo tanto:
#
# A * B
#
# es una multiplicación válida porque las dimensiones internas
# coinciden:
#
# (3×4)(4×10) = (3×10)
#
# En este ejemplo estamos construyendo manualmente el producto
# matricial columna por columna.
#
# Si:
#
# B = [b₁ b₂ b₃ ... b₁₀]
#
# entonces:
#
# AB = [Ab₁ Ab₂ Ab₃ ... Ab₁₀]
#
# Cada columna de la matriz resultado se obtiene multiplicando
# A por la columna correspondiente de B.
#
# Los tres métodos calculan exactamente el mismo resultado:
#
# C = AB
# D = AB
# E = AB
#
# Por ello:
#
# C - D = 0
# C - E = 0
#
# donde 0 representa una matriz de ceros.
#
# Este ejemplo es importante porque muestra una interpretación
# fundamental del producto matricial:
#
# "Multiplicar una matriz por otra equivale a multiplicar la
# primera matriz por cada columna de la segunda."
#
# Esta interpretación será utilizada posteriormente para
# transformar simultáneamente grandes conjuntos de puntos
# mediante una única multiplicación matricial.
```

In [ ]:
```
# Método 2 para multiplicar cada columna de la matriz B por la matriz A
# y almacenar los resultados en una matriz C de dimensiones correctas.

C = A * B    # Sí, es así de simple.
```

In [ ]:
```
# Utilice su método favorito para multiplicar cada columna de la matriz N
# por la matriz M y almacene el resultado en una matriz llamada myProd.

using Random
Random.seed!(101)
```

```
M = randn(11,11)
N = randn(11,11)

# myProd = ??

# Verificación de la solución
using LinearAlgebra

if isapprox(norm(myProd), 43.9059, atol = 1e-2)
    println("Buen trabajo. Está listo para comenzar el proyecto.")
elseif isapprox(norm(myProd), 35.5951, atol = 1e-2)
    println("Lo hizo al revés.")
    println("Multiplicó cada columna de M por la matriz N. Inténtelo nuev
else
    println("Si necesita ayuda, consulte al profesor o revise los ejemplo
end

# Recordemos la interpretación del producto matricial:
#
# N = [n₁ n₂ n₃ ... n₁₁]
#
# entonces:
#
# MN = [Mn₁ Mn₂ Mn₃ ... Mn₁₁]
#
# Es decir, multiplicar una matriz M por una matriz N equivale
# a multiplicar M por cada columna de N.
#
# En este ejercicio:
#
# M tiene tamaño 11×11
# N tiene tamaño 11×11
#
# por lo tanto:
#
# M*N
#
# produce otra matriz de tamaño 11×11.
#
# Observe que:
#
# M*N ≠ N*M
#
# en general.
#
# El orden de multiplicación importa.
#
# Esta propiedad será fundamental más adelante cuando construyamos
# transformaciones afines, donde el orden de aplicación de las
# transformaciones modifica el resultado final.
```

==========

TAREA 1 — TRANSFORMACIONES AFINES EN 2D

==========

En esta tarea graficará figuras simples y aplicará diferentes transformaciones geométricas.

El objetivo es aprender una versión matricial de lo que en matemática se denomina **transformaciones afines**.

También aprenderá una forma especial de escribir coordenadas que los especialistas en robótica utilizan con frecuencia: las **coordenadas homogéneas**.

In [ ]:
```
# Ejecute esta celda, no la modifique.
# En esta celda se crean algunas funciones auxiliares de graficación
# que serán utilizadas a lo largo del proyecto.


#=
    Esta función genera una gráfica 2D.

    Parámetros obligatorios:
        xdata -> coordenadas x
        ydata -> coordenadas y
        color -> color de la gráfica
=#
function plot_2D(xdata::Vector{Float64}, ydata::Vector{Float64}, color;
                 new=true, aspect_ratio=:equal,
                 xlims = (-2,2), ylims = (-2,2), grid=true,
                 framestyle=:origin, legend=false, linewidth=3)
    fxn = new ? plot : plot!
    fxn(xdata, ydata, seriescolor=color, aspect_ratio=aspect_ratio, xlims
        ylims=ylims, grid=grid, framestyle=framestyle, legend=legend,
        linewidth=linewidth)
end


#=
    Esta función genera una gráfica 2D de una nube de puntos.

    Parámetros obligatorios:
        xdata -> coordenadas x
        ydata -> coordenadas y
        color -> color de la nube de puntos
=#
function plot_2D_pointcloud(xdata::Vector{Float64}, ydata::Vector{Float64
                 new=true, seriestype=:scatter, markersize = 0.8,
                 markerstrokewidth = 0, dpi = 150, legend = false,
                 markercolor  = cgrad(:rainbow, rev = true))
    fxn = new ? plot : plot!
    fxn(xdata, ydata, seriestype=seriestype, markersize=markersize,
        markerstrokewidth=markerstrokewidth, dpi=dpi,
        markercolor=markercolor, marker_z=color, legend=legend)
end


#=
    Esta función genera una gráfica 3D de una nube de puntos.

    Parámetros obligatorios:
        xdata  -> coordenadas x
        ydata  -> coordenadas y
```

```
        zdata  -> coordenadas z
        limits -> tupla con los límites de los ejes
        color  -> color de la nube de puntos
=#
function plot_3D_pointcloud(xdata::Vector{Float64}, ydata::Vector{Float64
          limits, color::Vector{Float64};
           seriestype=:scatter3d, markersize=0.1, camera=(0,90), dpi=108
           markercolor=cgrad(:rainbow, rev=true), background_color=:blac
           foreground_color=:black, legend=false, aspect_ratio=1, new=fa
    fxn = new ? plot : plot!
    fxn(xdata, ydata, zdata, xlims=limits[1], ylims=limits[2], zlims=limi
    seriestype=seriestype, markersize=markersize, camera=camera, dpi=dpi,
    markercolor=markercolor, marker_z=color, background_color=background_
    foreground_color=foreground_color, legend=legend, aspect_ratio=aspect
    markerstrokewidth=0.001)
end


#=
    Esta función grafica el robot en 2D como una flecha.

    Parámetros obligatorios:
        xpos     -> coordenada x del robot
        ypos     -> coordenada y del robot
        limits   -> tupla con los límites de los ejes
        gradient -> orientación del robot (u,v)
=#
function plot_2D_robot(xpos::Vector{Float64}, ypos::Vector{Float64}, limi
        seriestype=:quiver, projection="3d", arrow=(10,10), camera=(0,90)
        background_color=:black, foreground_color=:black, legend=false, a
        seriescolor=:white, new=false)
    fxn = new ? plot : plot!
    fxn(xpos, ypos, seriescolor=seriescolor, seriestype=seriestype, proje
        gradient=gradient, arrow=arrow, xlims=limits[1], ylims=limits[2],
        camera=camera, dpi=dpi, background_color=background_color,
        foreground_color=foreground_color, legend=legend, aspect_ratio=as
end


#=
    Esta función grafica el robot en 3D.

    Parámetros obligatorios:
        xpos -> coordenada x del robot
        ypos -> coordenada y del robot
        zpos -> coordenada z del robot
=#
function plot_3D_robot(xpos::Vector{Float64}, ypos::Vector{Float64}, zpos
                    markersize=4, seriestype=:scatter3d, seriescolor=:whi
    fxn = new ? plot : plot!
    fxn(xpos, ypos, zpos, seriescolor=seriescolor, markersize=markersize,
end
```

In [ ]:

```
# Ejecute esta celda, no la modifique.
# En esta celda se crea un conjunto de puntos que, al ser graficados,
# forman un cuadrado.

using Plots
gr()
```

```
# default(fmt = :png)

#=
    Definimos los puntos que forman un cuadrado.

    Observe que se utilizan cinco puntos y no cuatro.

    Esto se debe a que el último punto coincide con el primero,
    permitiendo cerrar el contorno del cuadrado cuando se unen
    los puntos mediante segmentos de recta.
=#

# [x1 y1; x2 y2; ...; x5 y5]'
points = [0. 0.; 0. 1.; 1. 1.; 1. 0.; 0. 0.]';

# Después de la transposición, la matriz points tiene la forma:
#
# [x1 x2 ... x5;
#  y1 y2 ... y5]

# Extraemos las coordenadas x
x = points[1,:]

# Extraemos las coordenadas y
y = points[2,:]

# Graficamos los puntos junto con el sistema de coordenadas
plot_2D(x, y, :green)
```

Parte A: La primera operación que realizaremos sobre nuestro cuadrado es una traslación. La operación matemática correspondiente se denomina **traslación** (2 %)

Trasladaremos cada punto en **−0.5** unidades en la dirección $begin : math : textx$ $end : math : text$ y **−0.5** unidades en la dirección $begin : math : texty$ $end : math : text$, de modo que el cuadrado quede centrado alrededor del origen.

Su tarea: Aplicar una traslación

$$\begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

a los puntos que forman el cuadrado utilizando suma de vectores.

Posteriormente veremos cómo realizar esta misma operación mediante multiplicación matricial.

En este ejercicio:

$$\begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$$

Guarde los puntos trasladados en un arreglo llamado `points2` .Aquí presentamos tres operaciones muy importantes sobre puntos:

a) Traslación, que significa

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

donde t_x y t_y son constantes dadas. Aquí, x' representa una nueva coordenada x. No indica una transposición. Lo mismo aplica para y'.

b) Rotación de un objeto mediante un ángulo dado, que tiene la siguiente forma:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

donde θ es el ángulo mediante el cual deseamos rotar un conjunto de puntos en sentido antihorario.

c) Escalamiento de las coordenadas x e y de un conjunto de puntos para estirar o contraer un objeto, posiblemente en cantidades diferentes en las direcciones x e y. El escalamiento tiene la siguiente forma:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

A continuación estudiaremos estas operaciones una por una.

Lo primero que observamos es que la rotación y el escalamiento involucran multiplicación matriz-vector, mientras que la traslación involucra la suma de dos vectores.

Aunque la suma parece más sencilla que la multiplicación, en lo que podría parecer un uso algo perverso de las matemáticas, vamos a mostrar cómo convertir una traslación en una multiplicación matricial.

El resultado de este esfuerzo será una forma uniforme de representar estas tres operaciones, lo que simplificará la manipulación de aproximadamente 185 MB de datos LiDAR.

Parte A: La primera operación que realizaremos sobre nuestro cuadrado es una traslación. La operación matemática correspondiente se denomina **traslación**

Trasladaremos cada punto en **−0.5** unidades en la dirección x e y **−0.5** unidades en la dirección y, de modo que el cuadrado quede centrado alrededor del origen.

Su tarea: Aplicar una traslación

$$\begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

a los puntos que forman el cuadrado utilizando suma de vectores.

Posteriormente veremos cómo realizar esta misma operación mediante multiplicación matricial.

En este ejercicio:

$$\begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$$

Guarde los puntos trasladados en un arreglo llamado `points2`.

In [ ]:
```
# Definimos el vector de traslación t
t = [-0.5; -0.5]

# Recordemos que el vector points está organizado de la siguiente manera,
# donde cada columna contiene un par de coordenadas x-y que define uno
# de los vértices del cuadrado graficado anteriormente.
#
# points = [x1 x2 ... x5;
#           y1 y2 ... y5]

# Haga una copia de los puntos originales y llámela points2
points2 = copy(points)

# Su tarea: utilice el método de su preferencia para modificar los
# valores de points2 de modo que las coordenadas x-y almacenadas
# en él sean trasladadas por el vector t.

# SU CÓDIGO AQUÍ
# points2= points2 .+ t;

x2 = points2[1,:];
y2 = points2[2,:];

plot_2D(x2, y2, :steelblue, new=false)
```

In [ ]:
```
println("¡Si no pasa la prueba, puede volver atrás y corregir su trabajo!

# Verificación amigable
#
# Si el valor de is_it_correct_check1 es "Yes", entonces su respuesta
# podría ser correcta. Sin embargo, si el valor de is_it_correct_check1
# es "No", entonces su respuesta es definitivamente incorrecta.

is_it_correct_check1 = ((points2 - points) == hcat([t for i=1:size(points

@show is_it_correct_check1;
```

Parte B: Traslación como multiplicación matricial en coordenadas homogéneas

Nuestro objetivo aquí es mostrar cómo la operación de traslación puede escribirse como una multiplicación matriz-vector. Los especialistas en robótica desean hacer esto para que las tres operaciones —traslación, rotación y escalamiento— puedan escribirse exactamente con el mismo formato. Este pequeño trabajo adicional facilita mucho la manipulación de archivos LiDAR de gran tamaño.

Planteamos la siguiente pregunta: ¿cómo podemos usar la multiplicación matricial para lograr una traslación? La respuesta es que necesitamos utilizar un truco llamado **coordenadas homogéneas**. Esta es una operación simple sobre nuestros datos, porque solo necesitamos agregar un 1 a cada punto $[x; y]$. Es decir, convertimos $\begin{bmatrix} x \\ y \end{bmatrix}$ a coordenadas homogéneas mediante

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

La operación inversa, tomar un vector en coordenadas homogéneas y eliminar el 1, produce el vector original $[x; y]$. Esto parece casi demasiado simple como para mencionarlo, ¡pero lo hacemos de todos modos!

Afirmamos que podemos realizar una traslación vectorial multiplicando un vector en coordenadas homogéneas por la matriz

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Veamos si esto realmente funciona

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} * x + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} * y + \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} * 1 = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix},$$

donde hemos utilizado nuestro método de multiplicación matricial de columnas por filas. Puede rehacerlo usted mismo usando el método clásico de multiplicación matriz por vector para obtener el mismo resultado. Y cuando eliminamos el 1 para volver a colocar nuestro vector transformado en coordenadas cartesianas ordinarias, obtenemos

$$\begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix},$$

¡Listo! Está bien si esta operación no le resulta impresionante. Nos gusta construir el suspenso poco a poco.

¿Es necesaria esta afirmación aquí?

En general, una transformación afín 2D toma la siguiente forma

$$H_{3\times 3} = \begin{bmatrix} A_{2\times 2} & t_{2\times 1} \\ 0_{3\times 1} & 1 \end{bmatrix}.$$

Si A es una matriz de rotación, entonces H será una *transformación de cuerpo rígido*.

Una tarea que haremos por usted: implementar la traslación del cuadrado utilizando coordenadas homogéneas y la matriz de traslación presentada anteriormente.

Preste atención a esta parte, porque aquí aparece código que luego querrá copiar y modificar ligeramente cuando realice la rotación y el escalamiento en coordenadas homogéneas.

In [ ]:
```
# Definimos la matriz de traslación T
T = [1 0 -0.5; 0 1 -0.5; 0 0 1];

# Hacemos una copia de los puntos originales
points3 = copy(points);

# Agregamos un 1 a cada punto para construir coordenadas homogéneas
points3 = [points3; ones(1,5)];

# Aplicamos la traslación a cada uno de los puntos que forman el cuadrado
# multiplicando cada columna de points3 por la matriz T y almacenando
# nuevamente el resultado en points3
points3 = T * points3

#= Observación: podríamos haber utilizado un bucle for como en la Tarea 0

for i in 1:size(points3,2)           # Multiplica cada columna de
    points3[:,i] = T * points3[:,i]; # points3 por la transformación T
end

Sin embargo, points3 = T * points3 hace exactamente lo mismo porque

T * points3 = [ T*points3[:,1]  T*points3[:,2] ... T*points3[:,end] ]

La operación vectorizada termina siendo 2 veces más rápida y utiliza
7 veces menos memoria.

Puede utilizar el método que prefiera a medida que avance en este proyect
=#

# Extraemos las coordenadas x e y y descartamos el número 1
# al final del vector homogéneo

x3 = points3[1,:];
y3 = points3[2,:];

plot_2D(x3, y3, :black, new=false, linewidth=5)
```

Parte C: Rotación en Coordenadas Homogéneas (2 %)

Ahora utilizaremos la matriz de rotación 2D en coordenadas homogéneas:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Su tarea: Rotar nuestro cuadrado en el plano (x)-(y) un ángulo de 45 grados ((\pi/4)).

Construya la matriz de transformación necesaria para realizar la rotación del cuadrado.

In [ ]:
```
# Complete esta parte del código para realizar la multiplicación matricia
# correspondiente a la transformación. Modifique el arreglo EXISTENTE poi
# para que contenga el resultado de la transformación, tal como hicimos a
# NO cree un nuevo arreglo points3.
#
# Llame T a su matriz de transformación.
#
# Sugerencia: No reinicialice utilizando el cuadrado verde original;
# modifique directamente el arreglo points3 (cuadrado negro).
#
# Nota: Asegúrese de estar utilizando el cuadrado correcto
# (compárelo con el cuadrado negro mostrado en el PDF).
#
# Advertencia: No omita los términos coseno y seno en su matriz de transf
#
# Repetimos: el autocalificador asume que la matriz de transformación
# se llama T.
#
# SU CÓDIGO AQUÍ


points3 = T * points3

# Extraiga las coordenadas x e y y descarte el número 1 al final
x3 = points3[1,:]
y3 = points3[2,:]

plot_2D(x3, y3, :red, new=false, linewidth=5)
```

In [ ]:
```
# Verificación amigable
using LinearAlgebra

is_it_correct_check1 = abs(T[1,1] - cos(pi/4)) < 1e-4 ? "Yes" : "No"
is_it_correct_check2 = abs(1 - det(T[1:2,1:2])) < 1e-4 ? "Yes" : "No"
is_it_correct_check3 = abs(points3[1,2] + 0.707107) < 1e-4 ? "Yes" : "No"

@show is_it_correct_check1;  # Transformación
@show is_it_correct_check2;  # Transformación

if (is_it_correct_check1 == "No") || (is_it_correct_check2 == "No")
    println("Falló al menos una prueba sobre la matriz de transformación
    println("   a) No llamó T a su matriz de transformación.")
    println("   b) No utilizó las funciones trigonométricas adecuadas en
end

@show is_it_correct_check3;  # Vector de puntos
```

```
if is_it_correct_check3 == "No"
    println("Si falló la verificación de la transformación, probablemente
    println("Si pasó la verificación de la transformación, revise si util
    println("Si pasó la verificación de la transformación y utilizó el ve
    println("entonces verifique si guardó nuevamente los puntos transform
end
```

Parte D: Escalamiento

A continuación, vamos a escalar el cuadrado, lo que significa que "estiraremos" sus lados como si fuera una banda elástica. Los matemáticos a veces llaman a este tipo de transformaciones **no rígidas**.

La matriz de transformación de escalamiento en 2D está dada por:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Su tarea: Escalar el cuadrado rotado utilizando $S_x = 1.5$ y $S_y = 0.5$.

In [ ]:
```
# Complete esta parte del código para realizar la multiplicación matricia
# que implementa la transformación de escalamiento. En particular,
# modifique el arreglo points3 para que contenga el resultado de la
# transformación, tal como hicimos anteriormente.
#
# Nota: Asegúrese de estar utilizando el cuadrado correcto
# (compárelo con el cuadrado rojo mostrado en el PDF).
# Por lo tanto, su arreglo points3 debe contener el resultado correcto
# de la rotación y estar listo para ser modificado.

# SU CÓDIGO AQUÍ

# Extraiga las coordenadas x e y y descarte el último elemento
x3 = points3[1,:]
y3 = points3[2,:]

println("No necesita una celda de verificación porque puede")
println("comparar directamente su gráfica con la mostrada en el PDF del p

plot_2D(x3, y3, :blue, new=false, linewidth=5)
```

Parte E: Integrándolo todo: Transformaciones Afines (3 %)

Todas las transformaciones que hemos realizado hasta ahora —traslación, rotación y escalamiento—, así como sus combinaciones, se denominan **transformaciones afines**.

Cuando trabajamos con grandes conjuntos de datos y necesitamos aplicar una secuencia de traslaciones, rotaciones y escalamientos a un conjunto de puntos, puede imaginarse que llevar el control de todo el proceso de transformación puede volverse complicado con el tiempo.

La ventaja principal de utilizar la noción abstracta de una transformación afín es que podemos construir una transformación compleja multiplicando entre sí las transformaciones individuales consideradas una a una.

Por ejemplo, supongamos que queremos trasladar un conjunto de puntos por `[-0.5; -0.5]`, luego rotar el objeto trasladado 45 grados en sentido antihorario y, finalmente, escalar el objeto ya trasladado y rotado. ¡Son muchas transformaciones! Sí, pero podemos escribir la transformación total de manera tan simple como esta:

$$T = \begin{bmatrix} 1.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & -0.5 \\ 0 & 0 & 1 \end{bmatrix}$$

¿Qué hemos hecho? Preparamos la transformación homogénea total multiplicando tres transformaciones homogéneas individuales, que luego podemos aplicar todas de una sola vez a nuestro punto `[x;y]`, una vez que lo expresamos en coordenadas homogéneas como `[x;y;1]`.

¿Ya le explotó la cabeza?

Su tarea es implementar la secuencia anterior de transformaciones sobre el cuadrado original.

En la siguiente celda, construya la matriz de transformación `T` y luego aplíquela a nuestro vector `points`.

Haga lo que haga, no multiplique manualmente las matrices anteriores para ingresar una única matriz resultante. Escriba `T` como el producto de tres matrices homogéneas y deje que Julia realice la multiplicación matricial.multiplication for you.

In [ ]:
```
# Comenzamos creando una copia de los puntos originales y agregando
# un 1 a cada punto.
#
# Para uso posterior, observe la forma eficiente en que agregamos
# una fila de unos al arreglo points.

points4 = [copy(points); ones(1,5)]

# Complete esta parte del código para realizar la multiplicación
# matricial correspondiente a la transformación.
#
# En particular, modifique el arreglo points4 para que contenga
# el resultado de la transformación afín.
#
# Verificaremos tanto T como points4. No utilice una notación diferente.
#
# T vale 2 puntos y points4 vale 1 punto.
#
# T = ??
# points4 = ??


# Extraiga las coordenadas x e y y descarte el último elemento
x4 = points4[1,:]
```

```
y4 = points4[2,:]

println("Puede verificar su trabajo mediante esta gráfica.")
println("Si no obtiene el mismo rombo azul observado anteriormente,")
println("regrese y revise cuidadosamente sus cálculos.")

plot_2D(x, y, :green)
plot_2D(x4, y4, :blue, new=false, linewidth=5)
```

==========

TAREA 2 — NUBES DE PUNTOS EN 2D

==========

En esta tarea trabajaremos con nubes de puntos 2D. Este es el último paso antes de avanzar hacia las nubes de puntos 3D recolectadas con Cassie Blue. ¿Está empezando a sentir algo de ansiedad?

Una nube de puntos es simplemente una colección de puntos en el espacio (3D o superior) o en un plano (2D). Por lo tanto, para una nube de puntos 2D, las dos primeras filas de los datos contienen respectivamente las coordenadas (x) e (y) de cada punto. La tercera fila contiene información adicional sobre cada punto. En el caso de un sensor LiDAR, esta información corresponde a la intensidad; en el caso de una cámara, corresponde a los valores RGB (rojo, verde y azul).

Para esta nube de puntos 2D, piense en ella como si fuera una imagen de cámara. La nube de puntos está en un plano y los datos de color se entregan mediante una codificación RGB simplificada con valores entre 0 y 1. Puede imaginarlo como un arcoíris, donde el rojo tiene valor 0 y el azul tiene valor 1.

Se le proporciona una imagen distorsionada, lo que hace muy difícil leer el texto. Queremos aplicar una transformación afín para corregir la nube de puntos y hacerla legible.

Primero leeremos el contenido del archivo `question_image.csv` en un arreglo utilizando la función `readdlm`, y luego graficaremos los puntos mediante un gráfico de dispersión. Esta parte ya está proporcionada.

Para leer los datos, debe descomprimir el archivo de datos de modo que exista una carpeta llamada `data` en el mismo directorio que este notebook. De lo contrario, Julia no podrá acceder a los datos.

In [ ]:
```
using DelimitedFiles
using Plots
default(fmt = :png)

# NOTA IMPORTANTE: los datos fueron cargados como un archivo ZIP.
# Debe descomprimir la carpeta para poder acceder a los datos.
pointcloud = readdlm("data/question_image.csv", ',')
```

```
# Las coordenadas x e y corresponden a las primeras 2 filas de los datos
# y pueden extraerse del arreglo por separado.
point_data = pointcloud[1:2,:]

# Los datos de color de la nube de puntos forman la última fila
# y pueden guardarse por separado.
println("Observe el tamaño de los datos que está manipulando. Sí, esto es
println(" ")
println("El vector pointcloud[3,:] tiene $(length(pointcloud[3,:])) eleme

color_data = pointcloud[3,:]
```

In [ ]:
```
# Graficar la nube de puntos 2D utilizando los valores de color

plot_2D_pointcloud(
    point_data[1,:],   # coordenadas x
    point_data[2,:],   # coordenadas y
    color_data         # intensidad/color asociado a cada punto
)
# Graficamos la nube de puntos 2D.
#
# Cada punto posee:
#   - una coordenada x
#   - una coordenada y
#   - un valor de color/intensidad
#
# Los colores se representan mediante una escala tipo arcoíris:
#
#   0  -> rojo
#   ...
#   1  -> azul
#
# Observe que la imagen aparece distorsionada.
#
# En los próximos pasos construiremos una transformación afín
# capaz de corregir la geometría de la nube de puntos para
# recuperar la imagen original.
#
# Este es exactamente el mismo principio utilizado para corregir
# mediciones provenientes de cámaras, sensores LiDAR y sistemas
# de visión artificial.
```

Afortunadamente, se conoce la transformación homogénea que permite corregir la imagen:

$$T = \begin{bmatrix} -0.09239 & 0.038268 & 300 \\ -0.38268 & -0.923879 & 165 \\ 0 & 0 & 1 \end{bmatrix}$$

Su tarea: Aplicar (T) a la nube de puntos distorsionada y graficar la imagen corregida.

In [ ]:
```
X = [copy(point_data); ones(1,size(point_data,2))]; # Crear puntos
                                                    # homogéneos

#= Esta parte del código debe realizar lo siguiente:
```

```
1. Construir la matriz de transformación y almacenarla en la variable T.

2. Aplicar la transformación T a todos los puntos para obtener lo que
   llamamos una nube de puntos transformada.

   Puede utilizar la operación vectorizada o recorrer todos los puntos
   con un bucle, es decir, iterar desde el índice 1 hasta el número de
   columnas de X y multiplicar por la transformación.

3. Guardar los resultados directamente en el arreglo X.
   No cambie los nombres, porque las gráficas dependen de ellos.

4. Copie cuidadosamente los valores numéricos de T.
   Puede hacer doble clic sobre la celda Markdown y usar el mouse
   para copiarlos. Un solo dígito incorrecto puede hacer que falle
   una prueba.
=#

# T = ??
# X = ??
```

In [ ]:

```
# Verificación amigable
#
# Si el valor de is_it_correct_checkN es "Yes", entonces su respuesta
# podría ser correcta.
#
# Si el valor de is_it_correct_checkN es "No", entonces su respuesta
# es incorrecta.

is_it_correct_checkTa = isapprox(sum(T) - sum(abs.(T)), -2.79789799999997
is_it_correct_checkTb = isapprox(sum(T) + sum(abs.(T)), 932.076536, atol

if is_it_correct_checkTa & is_it_correct_checkTb
    println("La transformación T probablemente es correcta.")
    println(" ")
else
    println("Al menos un elemento de la matriz de transformación T es inc
    println("Si hace doble clic sobre la celda Markdown, incluso puede co
    println(" ")
end

# Ahora verificamos los datos almacenados en X.
#
# Los errores en X pueden deberse a:
#
# - una matriz T incorrecta;
# - una aplicación incorrecta de la transformación T sobre los datos;
# - haber almacenado los resultados en una variable distinta de X,
#   lo que ocasionará múltiples problemas en las siguientes celdas.

is_it_correct_check1 = abs((X[3, :] - ones(size(X, 2)))[1]) < 1e-4 ? "Yes
is_it_correct_check2 = abs(sum(X[2, :]) - -54512.825647621416) < 1e-4 ? "
is_it_correct_check3 = abs(sum(X[:,1]) - 164.00782569999998) < 1e-4 ? "Ye
is_it_correct_check4 = abs(sum(X[:, 5]) - 152.0378589000001) < 1e-4 ? "Ye
is_it_correct_check5 = abs(sum(X) - -181258.1626895815) < 1e-4 ? "Yes" :

@show is_it_correct_check1;
@show is_it_correct_check2;
@show is_it_correct_check3;
```

```
@show is_it_correct_check4;
@show is_it_correct_check5;
```

Verificación automática

Esta celda verifica que la transformación afín haya sido aplicada correctamente.

El autocalificador compara los primeros cinco puntos transformados con una solución de referencia utilizando una tolerancia numérica de `0.001`.

Si la celda se ejecuta sin errores, la transformación probablemente fue implementada correctamente.

Si aparece un error (`AssertionError`), revise:

- Los coeficientes de la matriz `T`.
- La construcción de las coordenadas homogéneas.
- La multiplicación `X = T * X`.
- Que el resultado haya sido almacenado nuevamente en la variable `X`.

In [ ]:
```
# Celda utilizada por el autocalificador.
#
# Esta celda compara los primeros cinco puntos transformados
# de su nube de puntos con una solución de referencia.
#
# No modifique esta celda.

test_x = [
    -2.003666599999974  -16.003373200000055  -15.004140000000007  -14.008
    165.01149229999996  164.06631959999987   163.98384749999994   163.993
    1.0                 1.0                  1.0                  1.0
];

@assert isapprox(X[:,1:5], test_x, atol = 0.001)
```

In [ ]:
```
# Ejecute esta celda únicamente si aprobó la verificación amigable,
# ya que la generación de la gráfica puede tardar algún tiempo,
# especialmente si muchas personas la están ejecutando simultáneamente.

plot_2D_pointcloud(X[1,:], X[2,:], color_data)

# Ahora visualizamos la nube de puntos después de aplicar
# la transformación afín correctiva.
#
# Si todo se realizó correctamente:
#
#   X = T * X
#
# la imagen distorsionada debería volverse legible.
#
# Observe que NO modificamos los colores originales.
# Solamente transformamos las coordenadas geométricas
# de cada punto.
#
# Conceptualmente:
#
```

```
# Nube original  -->  Transformación afín  -->  Nube corregida
#
# Este mismo principio se utiliza en:
#
# - procesamiento de imágenes
# - visión artificial
# - fotogrametría
# - robótica móvil
# - SLAM
# - reconstrucción 3D
#
# donde frecuentemente es necesario corregir distorsiones,
# cambiar sistemas de referencia o fusionar mediciones
# provenientes de diferentes sensores.
```

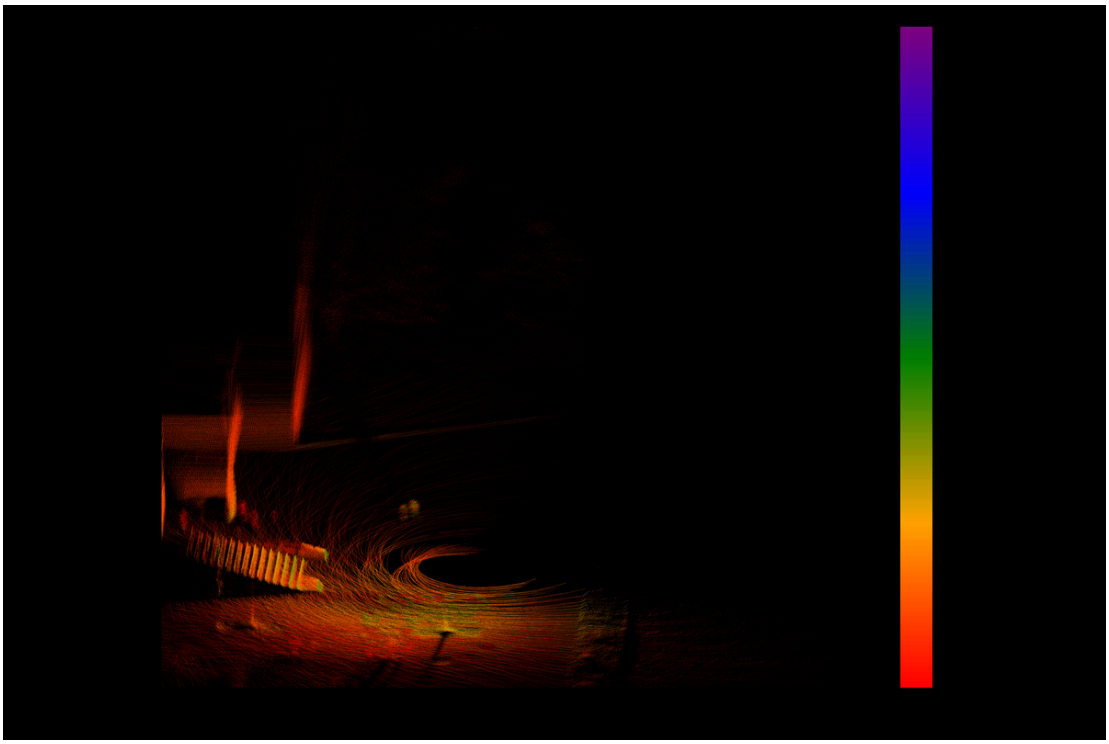
==========

TAREA 3 — CONSTRUCCIÓN DE MAPAS LiDAR 3D

==========

Como motivación, al finalizar esta tarea será capaz de generar una visualización similar a la mostrada a continuación.

¿Puede identificar qué lugar del Campus Norte de la Universidad de Michigan corresponde a esta escena?



En esta tarea trabajará con datos reales de un sensor LiDAR montado sobre el robot bípedo **Cassie Blue**.

El objetivo será comprender cómo transformar mediciones obtenidas en el sistema de referencia del sensor hacia un sistema de referencia global, para posteriormente

fusionar múltiples escaneos y construir un mapa tridimensional del entorno.

Este problema constituye una aplicación directa del Álgebra Lineal en robótica móvil, percepción espacial, visión computacional y sistemas autónomos.

Las mismas ideas matemáticas utilizadas en las Tareas 1 y 2 serán ahora aplicadas sobre cientos de miles de puntos tridimensionales provenientes de un sensor real.

Ahora que hemos visto el poder de las transformaciones matriciales en 2D, pasaremos al caso más desafiante en 3D. En este caso tampoco tendrá que preocuparse por la graficación, ya que el código correspondiente será proporcionado.

Su primera tarea consiste en graficar la nube de puntos sin procesar obtenida por el LiDAR de Cassie durante un período de 1 segundo. Como el LiDAR de Cassie opera a 10 Hz, se han recolectado 10 escaneos diferentes de datos LiDAR durante ese intervalo de un segundo y se han combinado en un único archivo grande. Al trabajar en este problema podrá observar el tamaño real de los datos y apreciar qué significa hacer matemática a escala.

Los datos recolectados están expresados en el sistema de referencia del LiDAR, el cual se mueve junto con el robot. Puede consultar el archivo PDF para ver cómo están organizados los datos.

Para simplificar la gestión de los datos —porque la gestión de datos no es el foco de ROB 101— hemos construido una función `data_parser`, que organiza la información proveniente de numerosos archivos CSV en distintos arreglos con significado específico. La función `data_parser` procesa cada segundo del conjunto de datos y lo convierte en cinco arreglos diferentes que necesitará utilizar en este problema.

La entrada de la función `data_parser` es el número identificador del intervalo de tiempo del conjunto de datos, con $id \in \{9, 10, \cdots, 13\}$, que identifica un conjunto de datos específico, es decir, el tiempo correspondiente. Este valor debe pasarse como argumento a la función, y los datos devueltos deben guardarse en el mismo orden en que son producidos por el parser. Consulte la primera celda de la Parte A para más detalles.

Los distintos arreglos que contendrán los datos son:

- `pointcloud_data`: contiene las coordenadas (x,y,z) de la nube de puntos.
- `Intensity_data`: contiene los datos de intensidad LiDAR, almacenados en el arreglo llamado `Intensity_data`.
- `R`: matriz de rotación.
- `t`: vector de traslación.
- `pose`: contiene la posición real del robot ((x,y,z)) en la columna 1 y la orientación del robot en la columna 2.

Muy pronto veremos un ejemplo de llamada a esta función en la Parte A.

In [ ]:
```
# Ejecute esta celda. No debe modificarla.
#
# Esta sección contiene funciones auxiliares que serán utilizadas
# durante toda la Tarea 3.
#
# Léala con atención:
#
#  • muestra cómo organizar código reutilizable;
```

```
#  • introduce el concepto de funciones;
#  • ilustra cómo procesar grandes volúmenes de datos;
#  • proporciona un ejemplo práctico del uso de transformaciones afines
#    sobre nubes de puntos LiDAR.
#
# En el Laboratorio 4 aprenderá a construir funciones similares.

using LinearAlgebra
using DelimitedFiles
using Plots
default(fmt = :png)

# ============================================================
# FUNCIÓN data_parser
# ============================================================
#
# Esta función carga y organiza automáticamente todos los datos
# asociados a un intervalo temporal de la trayectoria de Cassie.
#
# Entrada:
#
#     id ∈ {9,10,11,12,13}
#
# donde id identifica el segundo del recorrido que deseamos analizar.
#
# Salida:
#
#     pointcloud_data  -> coordenadas xyz de la nube de puntos
#     Intensity_data   -> intensidad LiDAR de cada punto
#     R                -> matriz de rotación
#     t                -> vector de traslación
#     pose             -> posición y orientación real del robot
#
# El objetivo es ocultar la complejidad de leer múltiples archivos
# CSV y permitir que el estudiante se concentre en el Álgebra Lineal.

function data_parser(id)

    # ------------------------------------------------------------
    # Carga de la nube de puntos LiDAR
    # ------------------------------------------------------------

    points = readdlm(
        string("data/cassie_data/Frame_",
               string(id),
               "/pointcloud.csv"), ',')

    pointcloud_data = points[1:3,:]    # coordenadas xyz
    Intensity_data = points[4,:]       # intensidad LiDAR

    # ------------------------------------------------------------
    # Carga de las transformaciones geométricas
    # ------------------------------------------------------------
    #
    # Estas matrices representan la transformación entre
    # distintos sistemas de referencia.
    #
    # Relacione este concepto con las transformaciones afines
    # estudiadas en las Tareas 1 y 2.
```

```
    Transformations = readdlm(
        string("data/cassie_data/Frame_",
               string(id),
               "/Transformations.csv"), ',')

    R = Transformations[:,1:3]   # matriz de rotación
    t = Transformations[:,4]     # vector de traslación

    # ------------------------------------------------------------
    # Carga de la pose real del robot
    # ------------------------------------------------------------
    #
    # Pose = posición + orientación

    pose = readdlm(
        string("data/cassie_data/Frame_",
               string(id),
               "/Position.csv"), ',')

    return (pointcloud_data,
            Intensity_data,
            R,
            t,
            pose)
end


# ============================================================
# EJEMPLO DE USO DEL PARSER
# ============================================================
#
# Se cargan los datos correspondientes al instante t = 10 s.

(pointcloud_data,
 Intensity_data,
 R,
 T,
 position) = data_parser(10)


# ============================================================
# FUNCIÓN affine_transform
# ============================================================
#
# Esta función aplica una transformación homogénea a una nube
# de puntos 3D.
#
# Observe que es exactamente el mismo procedimiento utilizado
# en las Tareas 1 y 2:
#
#   1. Homogeneizar los puntos agregando una fila de unos.
#   2. Aplicar una transformación afín.
#   3. Eliminar la coordenada homogénea.
#
# La diferencia es que ahora trabajamos con miles o cientos
# de miles de puntos simultáneamente.
#
# Esta es una de las ventajas fundamentales del Álgebra Lineal:
#
#     una sola multiplicación matricial
```

```
#
# puede transformar enormes conjuntos de datos.

function affine_transform(points, H)

    # Homogeneización
    transformed_p = [copy(points);
                     ones(1,size(points,2))]

    # Transformación afín
    transformed_p = H * transformed_p

    # Deshomogeneización
    return transformed_p[1:3,:]
end


# ============================================================
# FUNCIÓN robot_arrow
# ============================================================
#
# Función auxiliar utilizada únicamente para visualizar
# la orientación del robot en las gráficas.
#
# Extrae la dirección de avance a partir del ángulo yaw.
#
# No es necesario comprender los detalles internos para
# resolver este proyecto.

function robot_arrow(pose)

    position = pose[:,1]

    (roll,pitch,yaw) = pose[:,2]

    # Vector unitario asociado a la orientación del robot
    u = [1*cos(yaw-pi/2)]
    v = [1*sin(yaw-pi/2)]

    return (position, u, v)
end


# ============================================================
# PARÁMETROS DE VISUALIZACIÓN
# ============================================================
#
# Definen el tamaño de la ventana utilizada para graficar
# la nube de puntos y la trayectoria del robot.

window_size = 15

min_x = -window_size
max_x =  window_size

min_y = -window_size
max_y =  window_size

min_z = 0
max_z = 2*window_size
```

```
limits = (
    (min_x, max_x),
    (min_y, max_y),
    (min_z, max_z)
)
```

Ejecute esta celda, no la modifique.

Ejemplo de llamada a la función data_parser para obtener

los datos correspondientes al instante t = 13 segundos

de la recolección de datos.

Definimos el número identificador del intervalo de tiempo

id = 13

Procesamos los datos y los guardamos en distintos arreglos

(point_data, intensity_data, R, t, pose) = data_parser(id)

Observemos el tamaño de la nube de puntos para apreciar

la magnitud del problema que estamos resolviendo.

¿Puede imaginar seiscientas noventa y seis mil cuatrocientas

noventa y seis columnas?

Y eso corresponde solamente a un segundo de datos recolectados

por Cassie.

Ahora puede verse por qué la robótica necesita matemática

ejecutada a escala.

size(point_data)

==========

PARTE A — CARGA Y ORGANIZACIÓN DE LOS DATOS

==========

En esta sección se muestra cómo los datos son leídos desde los archivos y organizados en distintos arreglos para su posterior procesamiento.

Aunque no deberá modificar el código proporcionado, es importante comprender qué información contiene cada variable, ya que será utilizada en las siguientes partes del proyecto.

La próxima celda carga la nube de puntos correspondiente al último instante disponible del recorrido, ubicado en el segundo 13 (`id = 13`).

Este identificador temporal se pasa como argumento a la función `data_parser` , la cual devuelve automáticamente toda la información asociada a dicho instante.

Los datos obtenidos son:

- `pointcloud_data` : coordenadas tridimensionales (x, y, z) de la nube de puntos LiDAR.
- `Intensity_data` : intensidad registrada por el sensor para cada punto.
- `R` : matriz de rotación que describe la orientación relativa entre sistemas de referencia.
- `t` : vector de traslación asociado a la transformación geométrica.
- `pose` : estimación de la pose del robot, incluyendo posición y orientación.

Observe que los elementos `R` y `t` son exactamente los componentes necesarios para construir una transformación afín en 3D, generalizando las transformaciones estudiadas en las Tareas 1 y 2.

A partir de esta sección comenzaremos a trabajar con datos reales de un sensor LiDAR, aplicando los mismos conceptos de Álgebra Lineal utilizados anteriormente, pero ahora sobre nubes de puntos tridimensionales de gran tamaño.ranslation vector t to build an affine transformation, and finally an estimate of the robot's pose, that is, its position and orientation.

In [ ]:
```
# Ejecute esta celda, no la modifique.
#
# Ejemplo de llamada a la función data_parser para obtener
# los datos correspondientes al instante t = 13 segundos
# de la recolección de datos.

# Definimos el número identificador del intervalo de tiempo
id = 13

# Procesamos los datos y los guardamos en distintos arreglos
(point_data, intensity_data, R, t, pose) = data_parser(id)

# Observemos el tamaño de la nube de puntos para apreciar
# la magnitud del problema que estamos resolviendo.
#
# ¿Puede imaginar (696.496) seiscientas noventa y seis mil cuatrocientas
# noventa y seis columnas?
#
# Y eso corresponde solamente a un segundo de datos recolectados
# por Cassie.
#
# Ahora puede verse por qué la robótica necesita matemática
# ejecutada a escala.

size(point_data)
```

Como se mencionó anteriormente, los datos LiDAR se adquieren en el sistema de referencia del propio sensor mientras el robot está en movimiento.

Por lo tanto, cada escaneo LiDAR se obtiene desde una posición y orientación diferente del robot.

Una analogía útil es imaginar que usted camina mientras toma FOTOGRAFÍAS (no video) de un mismo entorno y luego intenta superponer todas esas imágenes sin tener en cuenta desde dónde fueron tomadas.

¿Qué esperaría obtener?

¿Una imagen coherente?

Probablemente no.

Lo más probable es que obtenga una imagen confusa, con objetos duplicados, desplazados o superpuestos incorrectamente.

Para destacar la importancia de conocer la posición y orientación de Cassie mientras se desplaza, vamos a superponer múltiples escaneos LiDAR sin transformarlos previamente a un sistema de referencia común, denominado sistema de referencia global (*world frame*).

Vamos a generar deliberadamente una visualización incorrecta.

Verá múltiples copias de los mismos objetos, como si estuviera viendo doble o incluso triple.

Esta experiencia permitirá comprender por qué las transformaciones geométricas son fundamentales en robótica móvil, percepción espacial y construcción de mapas.

La siguiente gráfica puede tardar algunos segundos en generarse.

In [ ]:
```
# Ejecute esta celda y tenga paciencia.
#
# Para acelerar la visualización, aquí se omite aproximadamente
# la mitad de los datos.

start_index = 9
end_index  = 13

plot()

for id = start_index:2:end_index

    println("Procesando Frame_", id, " ...")
    flush(stdout)

    # Guardar los datos en distintos arreglos
    (point_data, intensity_data, R, t, pose) = data_parser(id)

    # Graficar la nube de puntos 3D sin transformar.
    #
    # Observe que estos puntos todavía están expresados en el
    # sistema de referencia local del LiDAR.
    #
    # Como Cassie se mueve, superponer varios escaneos sin
    # transformarlos previamente al marco global producirá una
    # visualización desordenada.
    plot_3D_pointcloud(
```

```
        point_data[1,:],
        point_data[2,:],
        point_data[3,:],
        limits,
        intensity_data
    )
end

display(plot!())
```

Tenemos múltiples imágenes LiDAR de algo, pero resulta difícil decir con claridad qué es ese “algo”.

Para resolver este misterio, vamos a dar un paso atrás y procesar solamente un paquete de datos LiDAR antes de volver a trabajar con todos ellos en conjunto.

Como primer paso, debemos ser capaces de graficar dónde se encuentra nuestro robot en el sistema de referencia global.

In [ ]:
```
# Ejecute esta celda, no la modifique.
#
# Esta función construye una "flecha" para indicar la posición
# y orientación del robot dentro de una gráfica.

(point_data, intensity_data, R, t, pose) = data_parser(end_index)

(pos, u, v) = robot_arrow(pose);
```

In [ ]:
```
# Ejecute esta celda, no la modifique.
#
# Finalmente, vamos a representar la pose del robot mediante
# una flecha blanca.
#
# La flecha indica:
#
#   • la posición actual del robot (x, y)
#   • la dirección hacia la cual está orientado
#
# En las siguientes celdas superpondremos sobre esta misma
# gráfica la nube de puntos LiDAR.
#
# Esto permitirá visualizar simultáneamente:
#
#   1. la ubicación del robot,
#   2. su orientación,
#   3. las mediciones realizadas por el sensor.
#
# Esta combinación es fundamental para comprender cómo
# construir mapas del entorno a partir de datos LiDAR.

plot()

plot_2D_robot(
    pos[1,:],
    pos[2,:],
    limits,
```

```
    (u,v)
)
```

La flecha blanca indica la posición y orientación del robot.

Ahora que conocemos la pose de Cassie, también vamos a graficar la nube de puntos sin procesar en la misma figura.

El objetivo es observar si los anillos de la nube de puntos son consistentes con la pose de Cassie, es decir, con su posición y orientación.

In [ ]:
```
# Ejecute esta celda, no la modifique.
#
# Ahora vamos a visualizar la nube de puntos LiDAR sin procesar.
#
# Esta gráfica muestra exactamente lo que observa el sensor
# en el instante seleccionado.
#
# Tenga en cuenta que los datos todavía están expresados en el
# sistema de referencia local del LiDAR y no en el sistema de
# referencia global.
#
# Por esta razón, aunque la pose del robot ya es conocida,
# la nube de puntos aún no está alineada con el mundo.
#
# La generación de la gráfica puede tardar algunos segundos.
# Espere a que finalice el procesamiento.

id = end_index   # = 13

# Guardar los datos en distintos arreglos
(point_data, intensity_data, R, t, pose) = data_parser(id)

plot_3D_pointcloud(
    point_data[1,:],
    point_data[2,:],
    point_data[3,:],
    limits,
    intensity_data
)

display(plot!())
```

Observe cuidadosamente la gráfica que acaba de generar.

Como se explica en el PDF, el LiDAR está montado sobre el robot y, por lo tanto, **los círculos concéntricos que se observan en los puntos LiDAR deberían estar centrados alrededor del robot**.

Sin embargo, lo que vemos es que los datos están trasladados y rotados con respecto al robot.

Necesitamos corregir esta situación aplicando una transformación homogénea a la nube de puntos.

Transformación

Ha llegado el momento de corregir nuestro error.

Transforme la nube de puntos al sistema de referencia global (*world frame*) y grafíquela junto con la flecha de posición, tal como hicimos anteriormente. Luego compare ambas gráficas.

Para ello deberá construir una matriz de transformación afín a partir de la matriz de rotación (R) y del vector de traslación (t), que ya fueron proporcionados.

Sabemos que una transformación afín en 3D puede representarse como:

$$\begin{bmatrix} R_{3\times3} & t_{3\times1} \\ 0_{1\times3} & 1 \end{bmatrix}$$

donde:

- $R_{3\times3}$ es una matriz de rotación de tamaño 3×3.
- $t_{3\times1}$ es un vector de traslación de tamaño 3×1.
- $0_{1\times3}$ es un vector fila de ceros de tamaño 1×3.
- (1) es el número uno.

Transforme la nube de puntos y grafique la posición del robot como hicimos anteriormente.

¿Observa ahora círculos concéntricos LiDAR originándose desde la estimación de posición del robot?

In [ ]:
```
# Construya la transformación homogénea a partir de la matriz de
# rotación R y del vector de traslación t.
#
# Guarde el resultado en una matriz llamada H.
#
# Nota:
# Utilizamos el nombre H y no T para evitar confusiones con las
# transformaciones utilizadas anteriormente en las Tareas 1 y 2.
#
# Recordatorio:
#
#      [ R  t ]
# H =  [      ]
#      [ 0  1 ]
#
# donde:
#
#   R : matriz de rotación 3×3
#   t : vector de traslación 3×1
#
# H = ??
```

Before we go any further and use the transformation that you have built, let's check if you got it right! Please run the following **friendly checks** and then the

autograder cell. If you fail the tests in lines 5 through 8, then go back and see if you can fix your transformation matrix!

In [ ]:
```
# Verificación amigable
#
# Si el valor de is_it_correct_checkN es "Yes", entonces su respuesta
# podría ser correcta.
#
# Si el valor de is_it_correct_checkN es "No", entonces su respuesta
# es incorrecta.

is_it_correct_check1 =
    isapprox(
        H[:,1],
        [0.223755642141881,
        -0.968392974489662,
        -0.110220050665654,
         0.0],
        atol = 1e-3
    ) ? "Yes" : "No"

is_it_correct_check2 =
    isapprox(
        H[:,4],
        [2.52669466213533,
        -0.957786861851155,
         1.4354495810566,
         1.0],
        atol = 1e-3
    ) ? "Yes" : "No"

@show is_it_correct_check1;
@show is_it_correct_check2;# Ejecute esta celda, no la modifique.
#
# Utilizaremos la matriz de transformación H que acaba de construir
# y verificar.

# Finalmente, transformamos y graficamos los datos llamando a la función
# predefinida, generalizada a partir de la Tarea 1.
transformed_points = affine_transform(point_data, H)

# Visualización
plot()

plot_3D_pointcloud(
    transformed_points[1,:],
    transformed_points[2,:],
    transformed_points[3,:],
    limits,
    intensity_data
)

display("¿Puede ver ahora que los anillos LiDAR están más centrados alred
display("Paciencia. La gráfica se está generando.")

# Agregamos la pose del robot a la gráfica
plot_2D_robot(pos[1,:], pos[2,:], limits, (u,v))

display(plot!())
```

```
println("¿Puede ver ahora que los anillos LiDAR están más centrados alred
println("Ahora debería parecer que el robot está orientado hacia los dato
println("Anteriormente, podía parecer que el robot tomó algunos datos y l
```

In [ ]:

```
# Ejecute esta celda, no la modifique.
#
# Utilizaremos la matriz de transformación homogénea H que acaba
# de construir y verificar.
#
# El objetivo es transformar la nube de puntos desde el sistema
# de referencia del LiDAR hacia el sistema de referencia global
# (world frame).
#
# Para ello llamamos a la función affine_transform(), que implementa
# exactamente el mismo procedimiento estudiado en las Tareas 1 y 2:
#
#   1. Homogeneizar los puntos.
#   2. Aplicar la transformación H.
#   3. Eliminar la coordenada homogénea.
#
# Una vez realizada la transformación, visualizaremos:
#
#   • la nube de puntos transformada;
#   • la posición del robot;
#   • la orientación del robot.
#
# Observe cómo cambia la relación geométrica entre el robot
# y los datos LiDAR.

# Transformar la nube de puntos
transformed_points = affine_transform(point_data, H)

# Visualización
plot()

plot_3D_pointcloud(
    transformed_points[1,:],
    transformed_points[2,:],
    transformed_points[3,:],
    limits,
    intensity_data
)

display("¿Puede observar ahora que los anillos LiDAR están más centrados
display("Paciencia. La gráfica se está generando.")

# Agregar la pose del robot a la gráfica
plot_2D_robot(
    pos[1,:],
    pos[2,:],
    limits,
    (u,v)
)

display(plot!())

println("¿Puede observar ahora que los anillos LiDAR están más centrados
```

```
println("Ahora debería parecer que el robot está orientado hacia los dato
println("Anteriormente podía dar la impresión de que el robot había tomad
```

==========

PARTE B - CONSTRUCCIÓN DE MAPAS Y FUSIÓN DE ESCANEOS LiDAR

==========

Ahora comprendemos cómo pasar desde un sistema de referencia del sensor (*sensor frame*) hacia un sistema de referencia global (*world frame*). Comencemos entonces a construir un mapa fusionando datos obtenidos en múltiples instantes de tiempo.

Como habrá notado en la figura que acaba de generar utilizando un segundo de datos, los escaneos LiDAR recolectados durante un único segundo no son suficientes para proporcionarnos una comprensión completa de la escena. Por esta razón, necesitamos fusionar múltiples escaneos para obtener una representación más rica y detallada del entorno.

Aunque nos gustaría proporcionarle el conjunto completo de datos y que esta experiencia fuera tan impresionante como el video de Cassie, debido a restricciones de tiempo trabajaremos únicamente con cinco segundos de datos (desde el segundo 9 hasta el segundo 13), en lugar de los tres minutos completos de adquisición.

Sin embargo, una idea fundamental que debe extraer de este problema es que el robot se encuentra en movimiento continuo y, por lo tanto, debemos compensar dicho movimiento mediante una matriz de transformación apropiada que convierta los datos LiDAR expresados en el sistema de referencia del sensor a datos LiDAR expresados en el sistema de referencia global.

Esta transformación debe estimarse continuamente y en tiempo real sobre el robot.

De hecho, existe toda una línea de investigación dedicada al cálculo de las mejores transformaciones aproximadas que permiten relacionar un LiDAR en movimiento con un sistema de referencia global fijo.

¿Está convencido ya de que sería imposible realizar todo esto manualmente?

Para lograr que una computadora ejecutara estos cálculos con la velocidad necesaria, contamos con un equipo de estudiantes de posgrado que trabajó incansablemente durante meses. Y al final de todo ese esfuerzo, cerca de 50 estudiantes —casi todos de primer año— de la clase piloto de ROB 101 (*Go Pilot Class ROB-101!*) tomaron esos datos, los visualizaron y los manipularon para comprobar por sí mismos que nuestro algoritmo realmente funciona.

In [ ]:
```
## Parte B
start_index = 9
```

```
end_index  = 13

# Iterar sobre todas las nubes de puntos correspondientes
# a los intervalos de tiempo desde 9 hasta 13.
#
# En cada iteración:
#   1. se cargan los datos del intervalo actual;
#   2. se construye la matriz de transformación homogénea;
#   3. se transforma la nube de puntos al marco global;
#   4. se acumulan los puntos transformados en una nube final.
#
# El código puede tardar un tiempo en ejecutarse, así que tenga paciencia

final_points = [];
final_intensity = [];

for id = start_index : end_index

    transformed_points = [];

    #= Su solución debe realizar lo siguiente:

    1. Procesar los datos del intervalo de tiempo id utilizando
       la función data_parser y guardarlos en arreglos con los
       MISMOS NOMBRES utilizados anteriormente.

    2. Construir la matriz de transformación para ese intervalo
       utilizando los datos R y t.

    3. Transformar la nube de puntos y guardar el resultado en
       un arreglo llamado transformed_points.
    =#

    # SU CÓDIGO AQUÍ


end

@show final_points[:,1:5]
```

In [ ]:
```
# Celda del autocalificador
#
# Esta celda compara los primeros cinco puntos de la nube final
# transformada con una solución de referencia.
#
# No modifique esta celda.

final_points_test = [
    2.5354971032966303  7.711682136186845   7.789319078070248   3.0725850
    1.9722961774140295  14.224287690542685  11.201209438092262  3.3278194
    0.5592545760157881  0.8686608431526084  0.7348563412413707  0.5921945
];

@assert isapprox(final_points_test, final_points[:,1:5], atol = 0.0001)
```

In [ ]:
```
# La generación de esta gráfica puede tardar varios segundos.
#
# Atención:
# La nube de puntos fusionada contiene millones de coordenadas
```

```
# provenientes de múltiples escaneos LiDAR.
#
# Dependiendo de las características de su computadora, esta celda
# puede exigir una cantidad importante de memoria RAM y capacidad
# de procesamiento gráfico.
#
# Si experimenta lentitud, congelamientos o problemas de memoria,
# se recomienda utilizar la técnica de submuestreo (subsampling)
# estudiada anteriormente.
#
# Por ejemplo:
#
#   idx = 1:50:size(final_points,2)
#
# permite visualizar únicamente un punto cada 50 puntos,
# reduciendo significativamente el tiempo de renderizado.
#
# Otros ejemplos:
#
#   idx = 1:100:size(final_points,2)   # visualización más rápida
#   idx = 1:50:size(final_points,2)    # compromiso razonable
#   idx = 1:20:size(final_points,2)    # mayor detalle
#   idx = 1:10:size(final_points,2)    # alta densidad
#   idx = 1:1:size(final_points,2)     # todos los puntos
#
# En aplicaciones reales de robótica, visión artificial y LiDAR,
# el procesamiento eficiente de grandes volúmenes de datos es tan
# importante como la propia transformación geométrica.
#
# Si su equipo dispone de recursos suficientes, puede ejecutar
# la visualización completa.

plot()

# Actualizar la gráfica
plot_3D_pointcloud(
    final_points[1,:],
    final_points[2,:],
    final_points[3,:],
    limits,
    final_intensity'
)

display(plot!())
```

In [ ]:
```
# Para acelerar la visualización utilizaremos submuestreo.
#
# En lugar de representar todos los puntos de la nube fusionada,
# seleccionamos únicamente un punto cada 10 puntos.
#
# Esto reduce significativamente:
#   • el tiempo de renderizado,
#   • el consumo de memoria,
#   • la carga sobre la CPU y la GPU.
#
# La geometría global del mapa permanece prácticamente inalterada,
# pero la visualización resulta mucho más fluida.
#
# Si su computadora dispone de más recursos, puede aumentar la
# densidad visual utilizando:
```

```
#
#   idx = 1:5:size(final_points,2)
#
# o incluso:
#
#   idx = 1:1:size(final_points,2)
#
# para visualizar todos los puntos.
#
# Si experimenta lentitud, puede reducir aún más la carga mediante:
#
#   idx = 1:20:size(final_points,2)
#   idx = 1:50:size(final_points,2)
#   idx = 1:100:size(final_points,2)
#
# Esta técnica se denomina submuestreo (subsampling) y es ampliamente
# utilizada en robótica, visión artificial y procesamiento de nubes
# de puntos LiDAR de gran tamaño.

idx = 1:10:size(final_points,2)

plot()

plot_3D_pointcloud(
    final_points[1,idx],
    final_points[2,idx],
    final_points[3,idx],
    limits,
    final_intensity[idx]
)

display(plot!())
```

Hagamos un video

Excelente. Ahora ha construido un mapa de la escalera que aparece en nuestro video.

El siguiente paso consiste en iterar sobre todos los frames y generar un GIF, de modo que podamos observar cómo el mapa se va construyendo en tiempo real.

Simplemente copie el código de la pregunta anterior dentro del bucle que recorre todos los frames para construir el GIF.

Esta vez también agregaremos la posición del robot en la animación, representada como un punto blanco.

Completar el GIF vale cero puntos. No hay puntos que perder, pero sí mucha satisfacción que ganar.

In [ ]:
```
start_index = 9
end_index  = 13

Plots.plot()

# Construir una animación
```

```
anim = @animate for id = start_index:end_index

    transformed_points = [];

    # COPIE SU CÓDIGO DEBAJO


    # SU CÓDIGO AQUÍ
    #--------------------------------

    plot_3D_pointcloud(
        transformed_points[1,:],
        transformed_points[2,:],
        transformed_points[3,:],
        limits,
        intensity_data
    )

    # Truco auxiliar para graficar la posición
    if(id != start_index)

        # Graficar la posición anterior con un punto negro,
        # excepto en el primer frame
        (point_data, intensity_data, R, t, prev_pose) = data_parser(id-1)

        plot_3D_robot(
            [prev_pose[1,1]],
            [prev_pose[2,1]],
            [prev_pose[3,1]],
            seriescolor=:black
        )
    end

    plot_3D_robot(
        [pose[1,1]],
        [pose[2,1]],
        [pose[3,1]],
        seriescolor=:white
    )

    println("Porcentaje completado: $(100*(1+id-start_index)/(1+end_index

    prev_pose = copy(pose);

end

println("¡Generando el GIF!")
println("El punto blanco representa a Cassie moviéndose dentro del mapa,
println("el resto del mapa permanece fijo, como las escaleras, si realizó

gif(anim, "stairs_walking_five_seconds_zoomed_thirty.gif", fps = 1)
```

¿Lo has logrado ?

Sabemos que esa es una pregunta que probablemente quiera responder de inmediato.

No utilice Google Classroom ni el grupo de Telegram para consultar si una respuesta específica es correcta. Tampoco publique soluciones completas o fragmentos de

código de este trabajo práctico.

El trabajo debe realizarse de manera individual. Si necesita asistencia académica, puede consultar con el docente de la asignatura dentro de los espacios habilitados para ello.

A lo largo de este proyecto se han incorporado numerosas verificaciones amigables (*friendly checks*) que le permiten validar su trabajo paso a paso.

Gracias a estas verificaciones, debería haber podido confirmar gran parte de sus resultados antes de llegar a esta instancia final.

Si no obtiene todos los puntos, no se preocupe excesivamente.

El objetivo principal de este trabajo práctico no es alcanzar la perfección, sino comprender cómo los conceptos de Álgebra Lineal permiten resolver problemas reales de robótica, percepción y mapeo tridimensional.

Durante el desarrollo del proyecto tuvo la oportunidad de verificar resultados, corregir errores y fortalecer su comprensión de los conceptos estudiados.

Confíe en su razonamiento.

Cometer errores forma parte natural del proceso de aprendizaje.

Lo importante es identificar esos errores, comprender por qué ocurrieron y utilizarlos como una oportunidad para seguir mejorando.